Documentation & README: Data Path Design

Lab 4: RAM → PISO → SIPO → ALU Integration

Digital System Design Team

1 Project Overview

This repository contains the Verilog HDL implementation and verification of a streaming data path system[cite: 7]. The design stores 20-bit test vectors in a Parameterized RAM, converts parallel data into a serial bitstream using a Parallel-In Serial-Out (PISO) shift register, reassembles it back into a parallel word using a Serial-In Parallel-Out (SIPO) register, and finally decodes the fields to drive an Arithmetic Logic Unit (ALU)[cite: 7].

1.1 Dataflow Block Architecture

The flow of data through the system follows the pipelined stages:

1. **RAM**: Stores 20-bit test vectors containing the ALU Enable, Opcode, Operand A, and Operand B[cite: 7].
2. **PISO**: Fetches 20-bit words and serializes them MSB-first[cite: 7].
3. **SIPO**: Receives the serial stream using a self-timed `valid` → `shift_en` handshake[cite: 7].
4. **ALU**: Decodes the parallel word and executes arithmetic/logical operations[cite: 7].

—

2 System Architecture & Bit Mapping

2.1 SIPO Word Field Assignment

The reassembled 20-bit word output by the SIPO (`parallel_data[19:0]`) is split into control and operand fields for the ALU as detailed below[cite: 7]:

2.2 ALU Instruction Set Table

The ALU supports 6 operations specified by the 3-bit `opcode`[cite: 1, 7]:

—

Table 1: Bit Mapping of the 20-bit Parallel Word

Bit Range	Width	Signal Name	Description
[19]	1	alu_en	Master Enable for the ALU[cite: 7]
[18:16]	3	opcode	Operation Select Code[cite: 7]
[15:8]	8	in_a	First Operand (Operand A)[cite: 7]
[7:0]	8	in_b	Second Operand (Operand B)[cite: 7]

Table 2: ALU Opcodes and Operations

Opcode [2:0]	Instruction	Operation	Output (alu_out)
000	ADD	Addition	$in\_a + in\_b$[cite: 1, 7]
001	SUB	Subtraction	$in\_a - in\_b$[cite: 1, 7]
010	AND	Bitwise AND	$in\_a \,\&\, in\_b$[cite: 1, 7]
011	XOR	Bitwise XOR	$in\_a \oplus in\_b$[cite: 1, 7]
100	OR	Bitwise OR	$in\_a \mid in\_b$[cite: 1, 7]
101	OUT A	Pass-through Operand A	in_a[cite: 1, 7]
110--111	NOP	Default/Disabled	$8'b00000000$[cite: 1, 7]

3 Source Code Documentation

3.1 1. RAM Module (ram.v)

Listing 1: RAM Module Code

```
module ram #(parameter w1 = 8, dep = 256, w2 = 20)(
    input wire clk,
    input wire rst,
    input wire en1,
    input wire en2,
    input wire [w2-1:0] din,
    input wire [w1-1:0] addr,
    output reg valid,
    output reg [w2-1:0] dout
);

reg [w2-1:0] mem [0:dep-1];

always @(posedge clk or negedge rst) begin
    if(!rst) begin
        dout <= 0;
        valid <= 0;
    end
    else begin
        valid <= 0;
        if(en1)
            mem[addr] <= din;
        else if(en2) begin
```

```
                dout <= mem[addr];
                valid <= 1;
            end
        end
end

endmodule
```

3.2 2. PISO Shift Register (`piso.v`)

Listing 2: PISO Module Code

```
module piso #(parameter w2 = 20)(
  input wire clk, rst,
  input wire [w2-1:0] pin,
  input wire ram_valid,
  output reg en,
  output reg sout,
  output reg valid
);

reg [w2-1:0] shreg;
reg [4:0] counter;

always @(posedge clk or negedge rst) begin
  if(!rst) begin
    shreg   <= 0;
    sout    <= 0;
    valid   <= 0;
    en      <= 1;
    counter <= 0;
  end
  else begin
    if(ram_valid && en) begin
      shreg   <= pin >> 1;
      sout    <= pin[0];
      counter <= w2 - 1;
      valid   <= 1;
      en      <= 0;
    end
    else if(valid) begin
      sout    <= shreg[0];
      shreg   <= shreg >> 1;
      counter <= counter - 1;

      if(counter == 1) begin
        valid <= 0;
        en    <= 1;
```

```
37          end
38        end
39        else begin
40          en <= 1;
41        end
42      end
43  end
44
45  endmodule
```

3.3 3. SIPO Shift Register (pout.v)

Listing 3: SIPO Module Code

```
1  module pout #(parameter w = 20)(
2    input clk,
3    input rst,
4    input en,
5    input serialin,
6    output reg [w-1:0] pout
7  );
8
9  always @(posedge clk or negedge rst) begin
10   if(rst == 0) begin
11     pout <= 0;
12   end
13   else if(en == 1) begin
14     pout <= {serialin, pout[w-1:1]};
15   end
16 end
17
18 endmodule
```

3.4 4. ALU Module (alu.v)

Listing 4: ALU Module Code

```
1  module alu #(parameter w = 8, v = 3) (
2      input [w-1:0] in_a,
3      input [w-1:0] in_b,
4      input [v-1:0] opcode,
5      input en,
6      output reg [w-1:0] alu_out,
7      output reg a_is_zero
8  );
9
10     always @(*) begin
```

```
        a_is_zero = (in_a == {w{1'b0}});

        if (en) begin
            case(opcode)
              3'b000: alu_out = in_a + in_b;
              3'b001: alu_out = in_a - in_b;
              3'b010: alu_out = in_a & in_b;
              3'b011: alu_out = in_a ^ in_b;
              3'b100: alu_out = in_a | in_b;
              3'b101: alu_out = in_a;
              3'b110: alu_out = {w{1'b0}};
              default: alu_out = {w{1'b0}};
            endcase
        end
        else begin
            alu_out = {w{1'b0}};
        end
    end

endmodule
```

Listing 5: Top Module Code

3.5 5. Top Level Module (`top.v`)

```
module top #(
    parameter w1 = 8,
    parameter w2 = 20,
    parameter dep = 256
)(
    input wire clk,
    input wire rst,
    input wire en1,
    input wire en2,
    input wire [w1-1:0] addr,
    input wire [w2-1:0] din,
    output wire [7:0] alu_out,
    output wire azero
);

wire [w2-1:0] ram_dout;
wire ram_valid;
wire piso_en;
wire piso_valid;
wire serial_data;
wire [w2-1:0] parallel_data;

ram #(.w1(w1), .dep(dep), .w2(w2)) RAM1 (
```

```
24      .clk(clk), .rst(rst), .en1(en1), .en2(en2),
25      .din(din), .addr(addr), .valid(ram_valid), .dout(ram_dout)
26  );
27
28  piso #(.w2(w2)) PISO1 (
29      .clk(clk), .rst(rst), .pin(ram_dout), .ram_valid(ram_valid),
30      .en(piso_en), .sout(serial_data), .valid(piso_valid)
31  );
32
33  pout #(.w(w2)) SIPO1 (
34      .clk(clk), .rst(rst), .en(piso_valid),
35      .serialin(serial_data), .pout(parallel_data)
36  );
37
38  alu #(.w(8), .v(3)) ALU1 (
39      .in_a(parallel_data[15:8]),
40      .in_b(parallel_data[7:0]),
41      .opcode(parallel_data[18:16]),
42      .en(parallel_data[19]),
43      .alu_out(alu_out),
44      .a_is_zero(azero)
45  );
46
47  endmodule
```

—

4 Simulation Instruction for ModelSim

To observe all top-level and internal signals recursively during simulation, execute the following ModelSim command[cite: 6]:

```
add wave -r /top_tb/*; run -all
```